

Cambridge Dynamics

Design/Strategy Review

Section Liaison Officer

My Khe Nguyen

UR5 Team

Abi (subteam leader), Thara, Ian, Jose, Amanda, Christina

Mobile Robot Team

Karan (subteam leader), My Khe, Edward, Kirill, Jichi, Jared, Luana



UR5 - Software: Task Ordering

Task Selection

- Priority Queue Data Structure
- Weigh each task by (# of points)
 - * (stat. of dependencies)
 - Order of importance
- Operator input for task selection
- Adjust a weight to 0 to stop trying a task (for example, if points have been acquired or the logic is faulty)

Preliminary Roadmap

Heat Plate -> Place on Tray -> Pour Drink ->
Place on Tray -> Heat Bowl -> Place on Tray ->
Push Tray to Meal Ready Area -> Signal
"Ready" to Mobile Robot

Task Breakdown

Opening Microwave:

Grasp vertical handle (1) -> provide **counterforce** next to door (2)
-> **pull** along an **arc** (1)

Place Item in Microwave:

Grab and **lift** item (2) -> **place** on microwave plate (2)

Closing Microwave:

Grasp vertical handle (1) -> **push** along an **arc** (1)

Remove Item from Microwave:

Grab and **lift** item (2) -> move item to tray (2) -> **place** on tray (2)

Pouring Drink:

Grab bottle (1) -> **grab** cup (2) -> **lift** and **tilt** bottle to **pour** into cup (1) -> **move** cup to tray (2) -> **place** on tray (2)

If time permits: Stir Drink with a Spoon, Bimanual Manipulation



UR5 - Software: Robot and Operator Collaboration

Communication Options (Mobile Robot and UR5)

- Run both on the same program - not ideal, ideally should be decoupled
- HTTP server - a simple option, but less featureful
- ROS connectivity - more complex, but can tolerate more use cases

Key Strategies

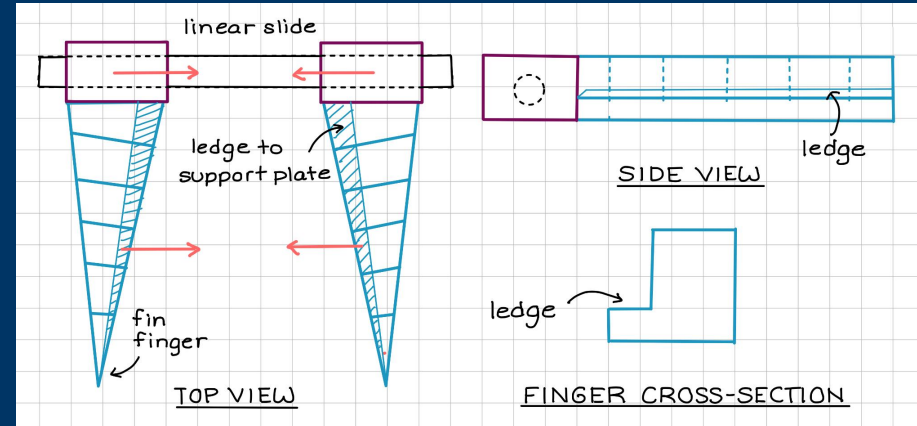
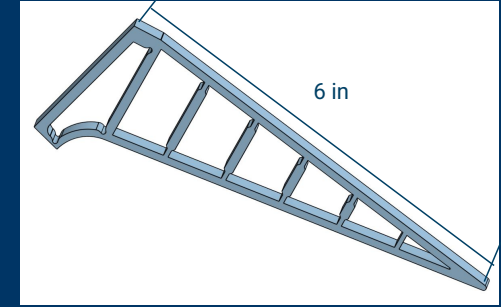
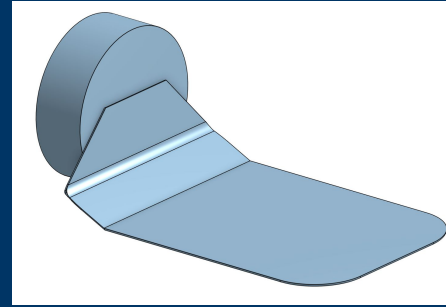
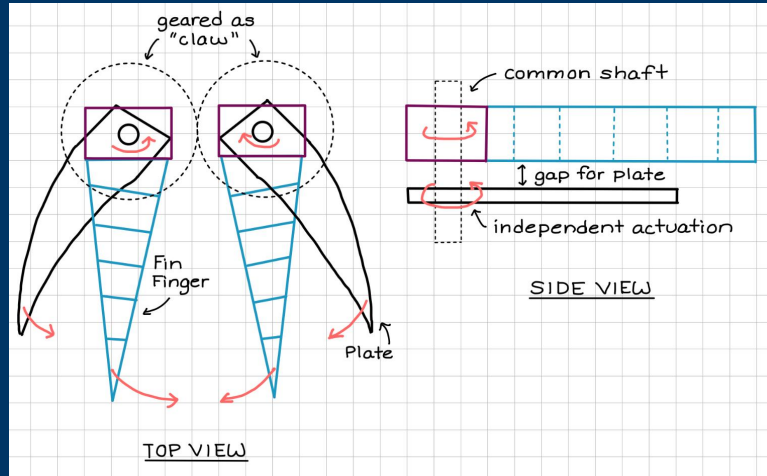
- Computer Vision for tray detection
- Operator task selection once the tray is ready

Robot Operator

- Break down tasks into subtasks
- Operator chooses next subtask
- Emergency stop for large issues
- Mode to switch to manual control



UR5 - Hardware



Distribution of Tasks

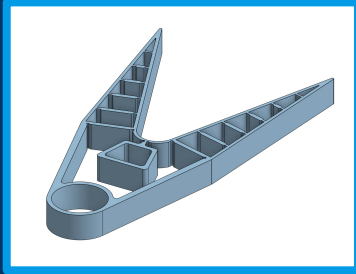
Arm 1: Manipulation of cup, bowl, and plate – large surface area, curvature >> **Fully custom gripper**

Arm 2: Manipulation of bottle, spoon, and microwave door handle - greater force concentrated in smaller surface area >> **Default gripper w/ compliant attachments**

UR5 - Hardware: Video Examples



UR5 - Hardware: Design Considerations



Advantages

- Compliance and dexterity provided by each fin while exerting clamping force
- Ability to grip objects with thin lips, and a variety of shapes like microwave handle

Drawbacks/predicted challenges

- Controlling actuator movement to apply the correct amount of force
- Limited normal force can be applied to hold object (weight considerations)

Fabrication and Actuation Method

- 3D Print (compliant material)
- Linear actuator to slide fins together or use motors to create “pinching” by rotating fins inward



Advantages

- Provides stability to complement the compliant fin gripper
- Simpler design for bimanual coordination
- Robust and able to bear force and weight

Drawbacks/predicted challenges

- Controlling orientation and position of spatula to ensure object is secured
- Risk of just pushing the object around

Fabrication and Actuation Method

- 3D Print (rigid material) or sheet metal
- Motor

Mobile - Software: Automation Strategy

Behavior-based autonomy

- Build a tree of modes that the vehicle could be in:
 - Go to prep area
 - Retrieve tray
 - Go to table 1
 - Deposit tray
 - Go to table 2
 - Retrieve tray
 - Go to dishwasher
- Each mode could have submodes
- Modes are determined by set of logic and state conditions
- Allows us to design behaviors for tasks independently

Mobile - Software: Considerations

State Estimation/Localization

- AprilTags for relative pose estimation to fixed known landmarks
- LIDAR/CV for object detection (dishwasher, ramp, tables, child)

Path Planning

- RRT for online planning
- Could potentially include vehicle dynamics to inform reasonable paths
- Min-jerk trajectories to reduce food movement

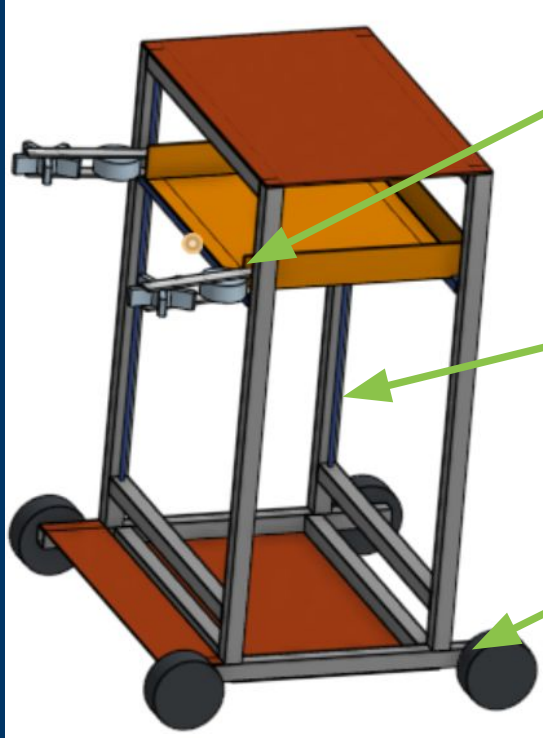
Control

- Control for tray handling: includes intake onto platform, elevator movement, and depositing to table/dishwasher
- Vehicle control: position, speed regulation

Collision Avoidance

- CV for detection of moving obstacle
- Estimate obstacle velocity by measuring distance traveled across subsequent frames

Mobile - Hardware: Designs



Task 3: Manipulating Trays

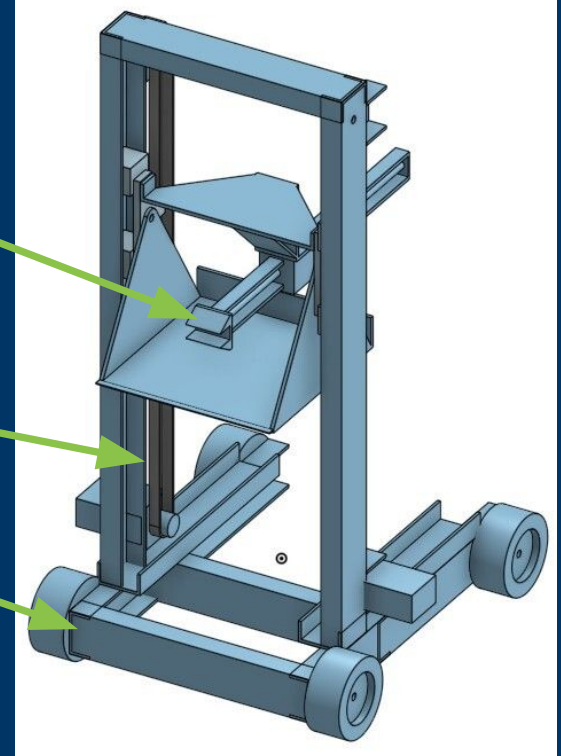
- Roller
- Prismatic arm

Task 2: Matching Height

- Lead screw + linear actuator
- Pulley elevator

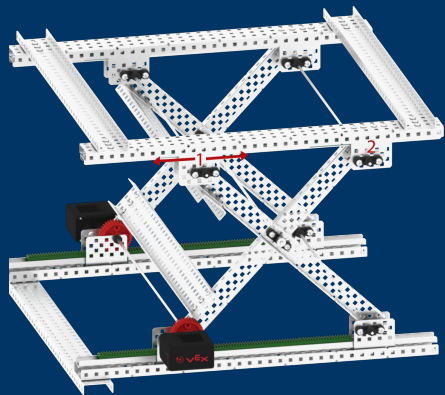
Task 1: Navigating the Field

- Base platform + camera + LIDAR



Mobile - Hardware: Design Considerations

Alternative mechanism



Outstanding questions

- Ramp traversing - would speed reduction work?
- Are tray size and location of tray on table fixed?

Implementation Detail

- Mobile base: default provided
- Structure/frame: aluminum
- Electronics/control: force sensor, LIDAR, camera
- Elevator:
 - If lead screw: lead screw, lead nut, stepper motor, shaft coupler, linear rails, bearing supports
 - If pulley: DC gearmotor, winch drum, cable, pulleys, linear rails, cable clamps, tensioner
- Grabbing mechanism: 3D printed parts, linear actuator

Backup Slides

Github Repository

Follow along with our progress!

[https://github.com/IanBallinger/
Robotics_final_spring2026_thursPM](https://github.com/IanBallinger/Robotics_final_spring2026_thursPM)



 **IanBallinger** a skeleton/sketch for the ur5 side of the project 0c77b8f · 24 minutes ago 8 Commits

URS	a skeleton/sketch for the ur5 side of the project	24 minutes ago
.gitignore	gitignore	2 days ago
.gitmodules	added https://github.com/mlt212/ur_2025.git examples	2 days ago
README.md	a skeleton/sketch for the ur5 side of the project	24 minutes ago
requirements.txt	a skeleton/sketch for the ur5 side of the project	24 minutes ago

 **README**  

Robotics_final_spring2026_thursPM

Note: ur_rtdc only works with python 3.12 on windows.

Getting Started:

from your terminal:

```
git clone https://github.com/IanBallinger/Robotics_final_spring2026_thursPM.git
cd Robotics_final_spring2026_thursPM
git submodule update --init --recursive
code .
```

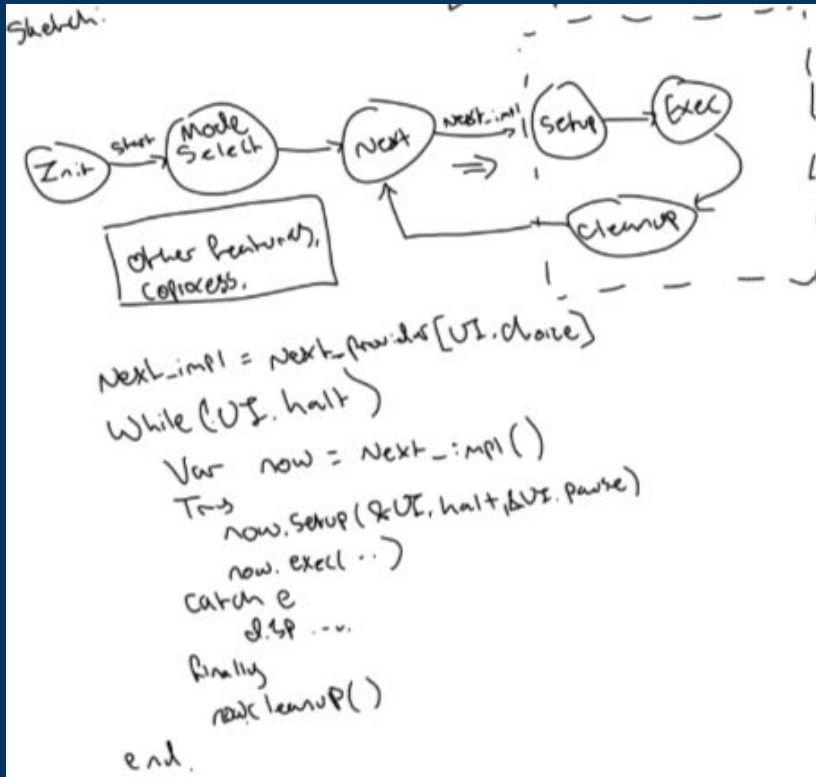
Then (windows): winget ships with win11, but you can also install it from [it's github releases page](#)

```
winget install Python.Python.3.12
py -3.12 -m venv venv
.\venv\Scripts\activate
pip install -r requirements.txt
```

(linux/macos): You should have python 3.12 installed.

```
python3.12 -m venv venv
source ./venv/bin/activate
pip install -r requirements.txt
```

UR5 - User Input and Task Encapsulation



```
if __name__ == "__main__":
    # Example usage
    class ExampleTask(UR5TaskInterface):
        """small example of UR5TaskInterface usage"""
        def setup(self):
            return True
        def perform_task_logic(self):
            return False
        def cleanup(self):
            pass

    task = ExampleTask(robot_ip="192.168.0.1")
    try:
        if not task.setup():
            raise RuntimeError("broken setup step")
        if not task.execute():
            raise RuntimeError("broken execute step")

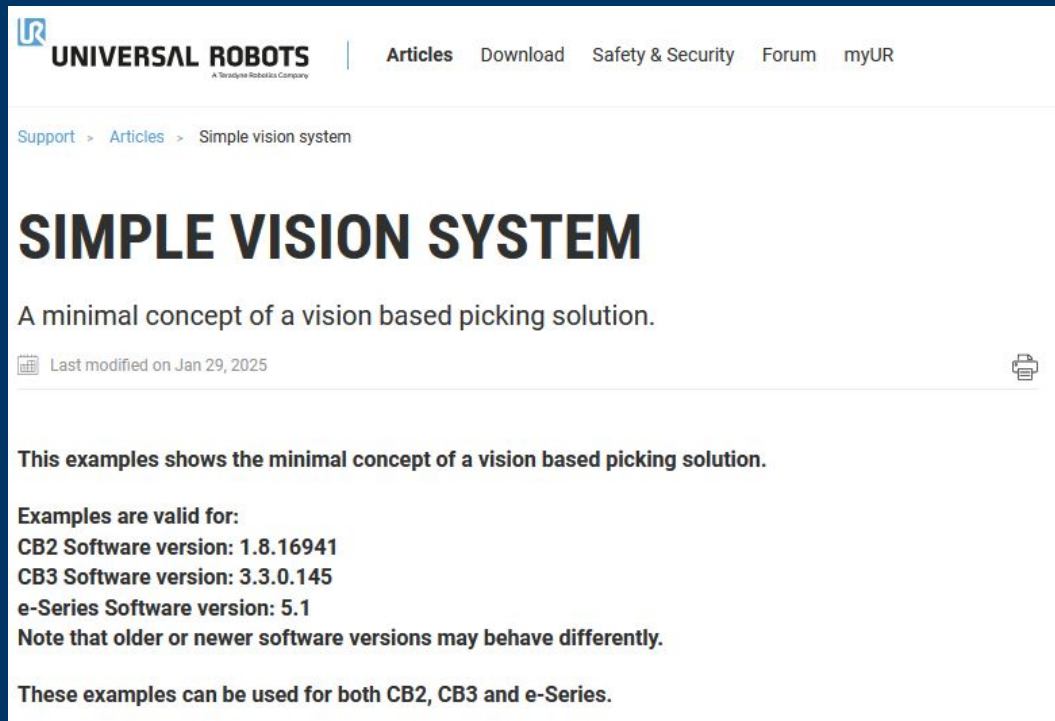
    except RuntimeError as error:
        print('Caught this error: ' + repr(error))

    finally:
        task.cleanup()

    sys.exit(0)
```

UR5 - Initial Item Localization

We will use the knowledgebase articles provided by UR to bootstrap this effort. (specifically, guidance on integrating vision systems into the robot controller)



The screenshot shows the Universal Robots Knowledgebase interface. At the top is the Universal Robots logo and navigation links: Articles, Download, Safety & Security, Forum, and myUR. Below the navigation bar is a breadcrumb trail: Support > Articles > Simple vision system. The main heading is 'SIMPLE VISION SYSTEM' in large, bold, black letters. Below the heading is a subheading: 'A minimal concept of a vision based picking solution.' To the right of the subheading is a printer icon. Below the subheading is a date stamp: 'Last modified on Jan 29, 2025'. The main body of the article contains the text: 'This examples shows the minimal concept of a vision based picking solution.' followed by a list of software versions: 'Examples are valid for: CB2 Software version: 1.8.16941, CB3 Software version: 3.3.0.145, e-Series Software version: 5.1'. A note follows: 'Note that older or newer software versions may behave differently.' At the bottom, it states: 'These examples can be used for both CB2, CB3 and e-Series.'

UNIVERSAL ROBOTS
A Teradyne Robotics Company

Articles Download Safety & Security Forum myUR

Support > Articles > Simple vision system

SIMPLE VISION SYSTEM

A minimal concept of a vision based picking solution.

Last modified on Jan 29, 2025

This examples shows the minimal concept of a vision based picking solution.

Examples are valid for:
CB2 Software version: 1.8.16941
CB3 Software version: 3.3.0.145
e-Series Software version: 5.1
Note that older or newer software versions may behave differently.

These examples can be used for both CB2, CB3 and e-Series.

<https://www.universal-robots.com/articles/ur/programming/simple-vision-system/>

UR5 - Elaboration on Priority Queue Data Structure

Task selection: [Priority queue data structure](#) (the canonical solution to this problem)

Key idea: Weight each task by (the number of points it enables) * (the status of its dependencies), and then do everything in order of importance.

Initial task list: [**heat plate**, heat bowl, pour drink, signal-ready to mobile bot]

Depends on: [(**available tray**, e-stop), (available tray, e-stop), (available tray, e-stop), (drink + bowl + plate)]

Task weights: [**6*1**, 6*1, 6*1, 4*0]

Each arm can **pick it's own task and run independently** based on hardware ability, tasks can be **collaborative**

Can be adjusted on the fly (i.e. there is a problem, adjust a weight to 0 to stop trying a task until it's fixed then change the number back) => a natural way for operator input to select tasks

From here: subtasks (arm1 movement to open microwave, arm2+arm1 movement to grab plate, ...)

Mobile - Software Requirements

- Largely similar to UR5
- OpenCV and AprilTag library for vision and pose estimation
- Existing behavior tree libraries like py_trees for the higher-level autonomy framework
- RTDE for intra-robot comms and comms to operator
- Control and planning are custom
 - Allows us to design and tune specifically for our platform specs

Mobile - Hardware: Chain lift



Mobile - Hardware: Prismatic Arm

